

35. Le dessin écrit là où on le pose

Les huit rangées dans le « poser » lui-même — sans nom, et sans numéro.

1. Ce qu'il faut comprendre

Une case de l'écran, c'est **huit rangées de huit pixels**, jamais autre chose. La leçon d'avant leur donnait un nom — `Tuile BLOC = {...}` — puis posait ce nom. Mais un nom, à l'arrivée, **c'est un numéro** : `poser(2, 2, BLOC)` écrit 44 dans la carte de fond.

Ces huit rangées peuvent s'écrire **dans le `poser` lui-même**. Le compilateur grave la tuile, lui trouve son numéro, et l'écrit à la case demandée — il n'y a plus ni nom à inventer ni numéro à suivre, et le dessin est **là où il apparaît**.

Ce n'est pas un remplacement, c'est un choix. Ce qui sert PARTOUT mérite son nom : le sol ci-dessous est posé vingt fois, et `SOL` le dit à chaque ligne. Ce qui ne sert QU'À UN ENDROIT n'apprend rien de plus en s'appelant `TUILE3`, et oblige à lire deux endroits pour en comprendre un seul.

Le même dessin ne se grave qu'une fois. Écris deux fois le même nuage à dix lignes d'écart : le compilateur reconnaît qu'il l'a déjà, et ne dépense qu'une tuile. Il reconnaît aussi un dessin déjà déclaré en `Tuile`, **à travers les deux écritures** — `"21111112"` et `"+-----+"` sont les mêmes pixels. La console n'en tient que 256 : les gaspiller ne se verrait qu'au jour où il n'y en aurait plus.

Cela marche aux quatre endroits où un numéro de tuile est attendu : `poser`, `poserPanneau`, `sprite`, et `sprite16` — qui veut alors **seize rangées de seize signes**, et les découpe lui-même en quatre tuiles. Ailleurs, des accolades restent une faute, et le compilateur le dit avec sa ligne.

Une chose que le dessin sur place ne fait pas : **il n'apparaît pas dans l'atelier**. Celui-ci lit les `Tuile NOM = {...}` du texte, et il ne saurait pas où réécrire un dessin qui n'a pas de nom. Ce qu'on veut peindre à la souris, on le nomme.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **472 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	62 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← <code>image()</code> — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre

```

AttendreAcces    12 o ← EcrireTexte  - attend de pouvoir écrire une case
AdresseCase     10 o ← poser()    - trouve l'adresse d'une case de l'écran
EcrireTexte      10 o ← texte()   - écrit un texte, lettre par lettre
CopierTuiles     94 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte     14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          160 o ← CopierTuiles - 8 lettres sur 43, et 2 tuiles dessinées
"sur place"      9 o ← texte()   - un texte gravé

```

Lettres gravées : ACELPRSU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

poser() × 3      25 o sur place, dans main() + AdresseCase (10 o)
texte() × 1      11 o sur place, dans main() + EcrireTexte (10 o)
image() × 1       3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <poser>    // pose une tuile sur une case du fond

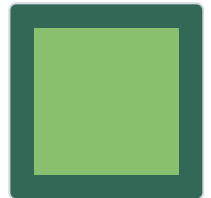
```

2. UN DESSIN — SOL

```

Tuile SOL = {
  "22222222",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "21111112",
  "22222222",
};

```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(4, 2, "SUR PLACE");

    // Le sol sert partout : il garde son nom.
    for (uint8_t x = 0; x < 20; x++) {
        poser(x, 12, SOL);
    }

    // La porte ne sert qu'ici : ses huit rangées sont ici.
    poser(9, 11, {
        "#####",
        "#-----#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#-###-#",
        "#####",
    });

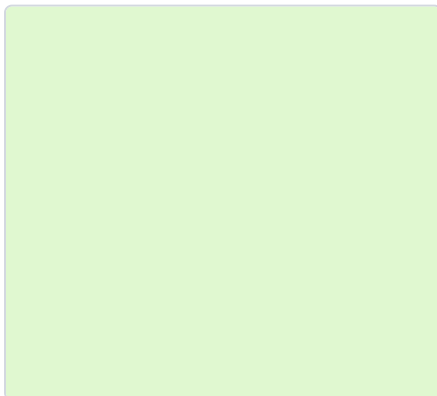
    // Le dessin de SOL, redit dans l'autre écriture : aucune tuile de plus.
    poser(3, 10, {
        "+++++++",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+-----+",
        "+++++++",
    });

    while (true) {
        image();
    }

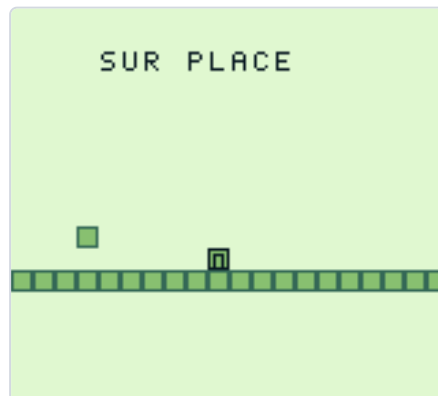
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un sol de vingt cases, une porte posée au-dessus, et à gauche un pavé qui reprend le dessin du sol.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ le sol posé par son nom couvre ses vingt cases
- ✓ la porte écrite sur place a sa propre tuile (tuile 45)
- ✓ et le dessin redit dans l'autre écriture retrouve SOL (tuile 44)

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++ [← l'atelier comp](#)

chercher un dessin : nom, étiquette, ★

 + tuile 8 × 8 + perso 16 × 16 + perso 32 × 32

Game Boy — 4 nuances Les quatre nuances de la console d'origine, et rien de plus. [Passer en couleur](#)

nuance 3
Nuances 

SOL tuile n° 44 — poser(x, y, SO

– zoom 22× + zoom [Plein écran](#)

[Peindre ✓](#) [Gomme](#) [/ Ligne](#) [Rectangle](#) [Cercle](#) [Formes : contour](#) [Remplir](#) [Pipette](#)

[Sélectionner et déplacer](#)

[Miroir](#) [↑↓ Miroir](#) [Tourner](#) [Copier le dessin](#) [Coller ici](#) [Dupliquer](#) [×2](#) [÷2](#) [Agrandir...](#)

[Vider le dessin](#)

Taille voulue : 

[✓ Créer la copie](#)

 taille réelle — 8 × 8 pixels

étiquettes

Animation
SOL ne bouge pas. Une animation toute prête (elle fabrique les images) :

[Eau \(vague\)](#) [Feu \(flamme\)](#) [Herbe \(vent\)](#) [Clignoter](#) [+ À la main](#)

Police
SOL peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

